

# pybmc: A Python package for Bayesian model combination

Troy Dasher<sup>1,2</sup>, Pablo Giuliani<sup>1</sup>, Kyle Godbey<sup>1</sup>, and An Le<sup>1,2¶</sup>

<sup>1</sup> Facility for Rare Isotope Beams, Michigan State University, East Lansing, MI 48824, USA

<sup>2</sup> Department of Physics and Astronomy, Michigan State University, East Lansing, MI 48824, USA ¶

Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

## Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

## Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

## Summary

Bayesian model combination (BMC) provides a principled framework for combining predictions from multiple models while quantifying uncertainty. Unlike simple averaging methods, BMC explicitly accounts for correlations among models and infers optimal model weights through Bayesian inference, producing predictions with calibrated uncertainty estimates.

The pybmc package implements this methodology in Python, allowing researchers to apply BMC to ensembles of predictive models defined on a shared domain. The package is designed for applications in which different theoretical or computational models produce systematically different predictions.

By providing a lightweight and flexible interface for constructing model ensembles and generating uncertainty-aware predictions, pybmc makes BMC accessible for research workflows. While motivated by applications in nuclear physics, the approach is general and applicable to a broad range of problems involving ensemble modeling and uncertainty-aware prediction.

## Statement of need

Across diverse scientific disciplines, it is increasingly common to encounter problems that rely on different theoretical models, which often yield systematically different predictions for the same observable. In such settings, the challenge is not only to compare models, but to combine them in a statistically meaningful and reproducible way that accounts for correlations, redundancy, and uncertainty. BMC offers a useful framework to address this problem by constructing weighted predictions from multiple models using dimensionality reduction and Bayesian inference.

Practical applications of BMC are limited by the lack of accessible and convenient software implementations. This software addresses this problem by providing a user-friendly, object-oriented framework Python package for BMC that supports the full workflow from aligned model tables to uncertainty-aware predictions. It integrates key steps such as data processing, dimensionality reduction (e.g., principal component analysis), and Bayesian statistical modeling into a coherent and reusable workflow. By organizing these operations into modular components, the package enables fast and simultaneous implementation of BMC across different datasets, while allowing flexible selection or extension of different components in the workflow.

## State of the fields

This package represents the first user-focused software implementation of the model orthogonalization and combination strategy detailed in (Giuliani et al., 2024). While existing

scientific Python tools such as scikit-learn (Pedregosa et al., 2011) and PyMC (Abril-Pla et al., 2023) provide components relevant to BMC, including dimensionality reduction and Bayesian inference, they do not offer an integrated implementation or any BMC workflow. There also exist other Bayesian model mixing software packages, including the Taweret software package (Ingles & others, 2024), but the current package is primarily designed to be used in scenarios where practitioners have precomputed databases of model predictions (with or without uncertainties) and wish to combine the results efficiently without needing to run additional simulations. This is particularly beneficial in nuclear physics, for instance, where global calculations of nuclear properties for all possible isotopes are computationally demanding.

## Bayesian Model Combination

BMC provides a framework for constructing a single predictive model from a collection of  $m$  existing models  $\mathcal{M}_k$ , each producing predictions  $M_k(x)$  for an observable defined over a common domain. The objective is to leverage the collective information contained in the ensemble while accounting for correlations and redundancies among models, which often arise when models share theoretical assumptions or calibration data.

We begin by assembling the predictions of the  $m$  models over  $n$  input locations into a matrix  $X \in \mathbb{R}^{n \times m}$ , where each column corresponds to a model and each row corresponds to a point in the domain. From this matrix, we define the ensemble mean prediction:

$$\phi_0(x_i) = \frac{1}{m} \sum_{k=1}^m M_k(x_i),$$

which captures the common structure shared across models. Subtracting this mean from the model predictions produces a centered representation that isolates the deviations between models.

To identify the dominant patterns in these deviations, we perform a singular value decomposition (SVD) of the centered matrix and retain a truncated set of components. This procedure yields an orthogonal basis  $\phi_j(x)$  that spans the principal directions of variability among the models, with  $j = 1, \dots, p$  with  $p \leq m$  controlling the effective dimensionality of the representation. This step reduces the dimensionality of the problem and mitigates the impact of redundant or highly correlated models by filtering out directions associated with minor variations.

The combined model is then constructed as a linear expansion in principal components:

$$f^\dagger(x; \mathbf{b}) = \phi_0(x) + \sum_{j=1}^p b_j \phi_j(x),$$

where  $\mathbf{b} = (b_1, \dots, b_p)$  are coefficients to be inferred from data. Because each basis function  $\phi_j$  is itself a linear combination of the original models, this representation is itself a (restricted) combination of the model ensemble.

The relationship between the combined model and experimental observations  $y_i$  is described through  $y_i = f^\dagger(x_i; \mathbf{b}) + \epsilon_i$ , with  $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ , where  $\sigma$  represents the typical scale of the discrepancy between the model and the data and is inferred within the Bayesian framework.

Given a set of observations  $\mathbf{y}$ , the likelihood function takes the form:

$$p(\mathbf{y} \mid \mathbf{b}, \sigma) \propto \prod_{i=1}^n \frac{1}{\sigma} \exp \left( -\frac{(f^\dagger(x_i; \mathbf{b}) - y_i)^2}{2\sigma^2} \right).$$

Combining this likelihood with suitable prior distributions for  $\mathbf{b}$  and  $\sigma$  yields the posterior distribution  $p(\mathbf{b}, \sigma \mid \mathbf{y})$ , which is sampled in pybmc using conditional Gibbs updates for the coefficients and Metropolis–Hastings updates for the variance parameters.

Predictions at new inputs are obtained by evaluating the combined model across posterior samples, resulting in a posterior predictive distribution:

$$p(y(x) \mid \mathbf{y}) = \int p(y(x) \mid \mathbf{b}, \sigma), p(\mathbf{b}, \sigma \mid \mathbf{y}), d\mathbf{b}, d\sigma,$$

84 from which summary statistics such as median predictions and credible intervals can be  
85 computed.

86 This formulation allows the BMC approach to reduce the effective dimensionality of the model  
87 space through orthogonalization, limit the impact of correlated or redundant models, and  
88 produce uncertainty-calibrated predictions. A more detailed discussion of the methodology can  
89 be found in (Giuliani et al., 2024).

## 90 Software design

91 pybmc is designed to handle precomputed model predictions, allowing researchers to combine  
92 existing calculations without modifying or rerunning the underlying simulators. This approach is  
93 useful when simulations are computationally expensive or available only as published prediction  
94 tables. It also narrows the package's scope into handling data and making BMC predictions.

95 Therefore, pybmc is organized around two core classes that separate data management from  
96 Bayesian inference, supported by small utility modules that implement the numerical algorithms.

97 The Dataset class (module pybmc.data) encapsulates all data access and preprocessing. It  
98 loads model outputs and (optionally) experimental truth values from HDF5 or CSV files into  
99 synchronized, pandas DataFrames. Users specify a list of model identifiers, a list of physical  
100 properties (e.g., binding energy or charge radius), and the domain columns (e.g., nucleon  
101 numbers N and Z). Dataset.load\_data then aligns the models on their common domain  
102 and, when a truth\_column\_name is supplied, left-joins the truth data so that experimental  
103 information can live on a smaller domain than the model ensemble. Additional methods provide  
104 convenient views of the loaded data, domain-aware filtering, and utilities for constructing  
105 training/validation/test splits.

106 The BayesianModelCombination class (module pybmc.bmc) implements the BMC workflow.  
107 It operates on the synchronized DataFrames produced by Dataset and keeps track of the  
108 list of models to be combined and the column containing the experimental truth. The class  
109 exposes three main operations: orthogonalize, which constructs an orthogonal basis of model  
110 differences by applying singular value decomposition (SVD) to centered model predictions;  
111 train, which performs Gibbs sampling on the truncated SVD coefficients to obtain posterior  
112 samples of the combination weights and residual uncertainty; and predict / evaluate, which  
113 turn those samples into posterior predictive draws, credible intervals, and coverage diagnostics.

114 Low-level numerical routines are factored into the pybmc.inference\_utils and  
115 pybmc.sampling\_utils modules. The former provides SVD post-processing (USVt\_hat\_extraction)  
116 and a Gibbs sampler for Bayesian linear regression, while the latter provides utilities to turn  
117 posterior samples into predictive draws and credible intervals (rndm\_m\_random\_calculator)  
118 and to compute coverage statistics (coverage). Keeping these routines outside the main  
119 classes keeps the public API small and makes it straightforward to swap or extend samplers  
120 and diagnostic tools without altering the high-level workflow.

121 The package is deliberately designed to be domain-agnostic, so that the same architecture  
122 can be applied to any application where practitioners need to combine multiple, potentially  
123 correlated predictive models into a single, uncertainty-quantified prediction.

124 More detailed tutorials, additional examples, and a complete API reference are available in the  
125 online documentation:

126 <https://ascsn.github.io/pybmc/docs>

## 127 Research impact statement

128 Would like some more use cases here

129 The original methodology was published in a peer-reviewed journal (Giuliani et al., 2024),  
130 and the first application of the methodology has also recently been published in an analysis  
131 of new experimental data from the Facility for Rare Isotope Beams (Ireland et al., 2026).  
132 Once the package was completed, it began replacing the original, bespoke implementation in  
133 new scientific works, including a study on Q-alpha trends in superheavy nuclei, the impact  
134 of nuclear uncertainties in r-process nucleosynthesis, and a systematic study of charge radii.  
135 Students involved in these projects have presented results obtained using the software at the  
136 American Physical Society's annual Division of Nuclear Physics meeting and at presentations  
137 locally. The package has also been adopted for release with the Bayesian Analysis for Nuclear  
138 Dynamics collaboration's (BAND) V0.5 release (Beyer et al., 2023) and disseminated broadly  
139 to the nuclear physics community.

## 140 AI usage disclosure

141 Generative AI was used in the creation of a package template early in the development process  
142 and during development via inline autocomplete within the developers' editors. No code was  
143 adopted without human oversight, and our metric for correctness was defined by detailed  
144 numerical comparison to the bespoke implementation already published in (Giuliani et al.,  
145 2024). AI tools were also used to expand the package documentation and write additional tests,  
146 with the resulting documentation going through a round of human review for completeness,  
147 correctness, and accessibility.

## 148 Acknowledgments

149 This needs done

## 150 Citations

- 151 Abril-Pla, O., Andreani, V., Carroll, C., Dong, L., Fomesbeck, C. J., Kochurov, M., Kumar,  
152 R., Lao, J., Luhmann, C. C., Martin, O. A., Osthege, M., Vieira, R., Wiecki, T., & Zinkov,  
153 R. (2023). PyMC: A modern and comprehensive probabilistic programming framework in  
154 Python. *PeerJ Computer Science*, 9(e1516). <https://doi.org/10.7717/peerj-cs.1516>
- 155 Beyer, K., Buskirk, L., Chan, M. Y.-H., Chang, T. H., DeBoer, R. J., Furnstahl, R. J., Giuliani,  
156 P., Godbey, K., Ingles, K., Liyanage, D., Nunes, F. M., Odell, D., Phillips, D. R., Plumlee,  
157 M., Pratola, M. T., Sempowski, A. C., Sürer, Ö., Wild, S. M., & Yannotty, J. C. (2023).  
158 *BANDFramework: An open-source framework for Bayesian analysis of nuclear dynamics*  
159 (Version 0.3.0). <https://github.com/bandframework/bandframework>
- 160 Giuliani, P., Godbey, K., Kejzlar, V., & Nazarewicz, W. (2024). Model orthogonalization and  
161 bayesian forecast mixing via principal component analysis. *Phys. Rev. Res.*, 6, 033266.  
162 <https://doi.org/10.1103/PhysRevResearch.6.033266>
- 163 Ingles, K., & others. (2024). Taweret: A python package for bayesian model mixing. *Journal*  
164 *of Open Source Software*, 9(97), 6175. <https://doi.org/10.21105/joss.06175>
- 165 Ireland, C., Bollen, G., Campbell, S., Chen, X., Erington, H., Gamage, N., Godbey, K., Houff,  
166 A., Izzo, C., Knight, B., Lalit, S., Leistenschneider, E., Lykiardopoulou, E., Maier, F.,  
167 Nazarewicz, W., Orford, R., Porter, W., Quick, C., Ravlic, A., & Villari, A. (2026). Mass  
168 of sn 101 and bayesian extrapolations to the proton drip line. *Physical Review C*, 113.  
169 <https://doi.org/10.1103/vck7-1c4t>

- 170 Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M.,  
171 Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,  
172 Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python.  
173 *Journal of Machine Learning Research*, 12, 2825–2830.

DRAFT